

NV-Segment-CTMR 模型简介

从 CT / MRI 到三维器官标签

NV-Segment-CTMR 学习笔记

面向入门学习者的简明讲解

2026 年 9 月

它解决什么问题？

输入

三维 CT / MRI
每个 voxel 是图像强度



模型

自动寻找结构
器官、组织等类别



输出

三维标签体数据
每个 voxel 一个标签

- 支持 CT 与 MRI（身体和脑部），用于**自动分割**
- 官方类别表包含 **345+ 类**；不同模态使用的类别集合并不完全相同
- 没有属于目标结构的位置会保留为 background

输入是图像强度，输出才是器官标签。

一张图看懂整体流程

1. Preprocessing

统一方向、spacing 与强度，让不同扫描更容易比较。

2. Sliding Window

把很大的三维影像分成 GPU 能处理的小块。

3. 3D CNN

编码图像，再逐步恢复空间位置与细节。

4. Feature Map

把灰度变成描述边缘、形状和位置的多维特征。

5. Class Matching

图像特征与器官的 class embedding 进行匹配。

6. Merge & Output

融合重叠窗口，恢复坐标并生成最终标签。

先整理数据，再提取特征、寻找类别，最后拼回完整结果。

Preprocessing: 先把数据整理好

Orientation | 统一方向

把影像调整到一致的坐标方向，避免左右、前后定义混乱。

Crop | 裁掉大块背景

保留包含身体的区域，减少没有信息的计算。

Spacing | 统一物理尺度

重采样到统一的 voxel 尺度，让相同器官看起来大小更接近。

Intensity | 处理强度

缩放并截断极端值，让不同扫描的数值范围更稳定。

预处理只是在“整理试卷”，真正的器官识别还没有开始。

训练：模型如何一步步变准？

- ① **前向传播**：输入影像，模型按当前参数生成预测 mask
- ② **比较**：预测与人工标注的 ground-truth mask 产生 loss
- ③ **反向传播**：计算各参数对错误应负多少责任
- ④ **更新**：optimizer 根据梯度修改参数，再重复训练

可以记成五个词

尝试 → 比较



修改 ← 追责



再尝试

训练就是不断“先做题、看答案、找原因、改方法”。

Sliding Window: 为什么要切成小块？

主要原因：显存有限

完整三维 CT / MRI 往往很大，难以一次全部送入 GPU。滑窗让模型一次只处理一个 3D patch。

为什么窗口要重叠？

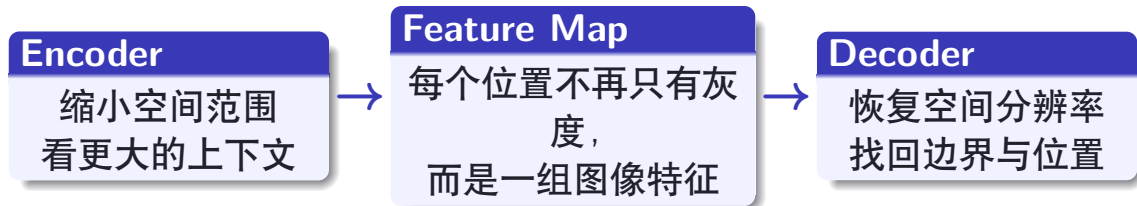
同一区域会被多个窗口预测，再把结果融合，可减轻 patch 边缘的不稳定与断裂。

最容易讲错的一点

滑动窗口不是为了让 patch 直接“互相交流”，而是为了控制单次显存；重叠主要用于减少块边缘伪影。

- 整幅图基本仍要被扫过，重叠区域还会重复计算
- 所以它主要降低**单次内存需求**，不一定降低总计算量

3D Encoder-Decoder 与 Feature Map



- 这些特征可以描述边缘、纹理、形状、位置和周围结构
- 本模型采用 SegResNet 式 3D 编码器-解码器；中间特征图会缩小，再由 decoder 恢复空间位置

Feature Map 是模型整理出的“线索地图”，还不是最终答案。

Class Embedding: 告诉模型要找什么

Image Feature

“这个位置看起来有什么？”

来自 3D 图像的特征

Class Embedding

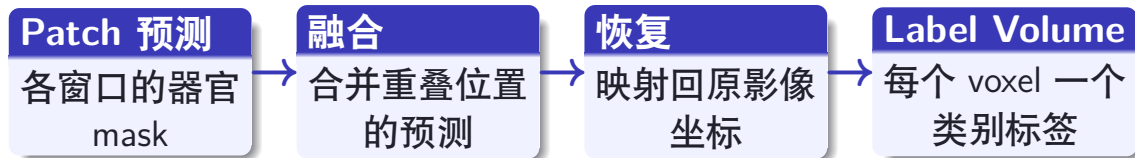
“现在要寻找哪一类？”

每一类都有一个可学习编码

→ 匹配后得到该类别的分数与 mask

- 模型通过类别 ID 选择相应的 class embedding
- 图像特征与类别编码匹配，为所选模态的支持类别生成 mask
- 它学到的是**类别编码与图像特征的对应关系**，不是词语的字面含义

最终输出： 三维 Label Volume



- 最终输出不是 feature vector，而是器官 / 组织标签或 background
- 标签体数据可用于着色、统计体积，并进一步生成三维表面模型

滑窗解决“图像太大”；Feature Map 说明“看到了什么”；
Class Embedding 说明“要找什么”；Label Volume 给出最终答案。